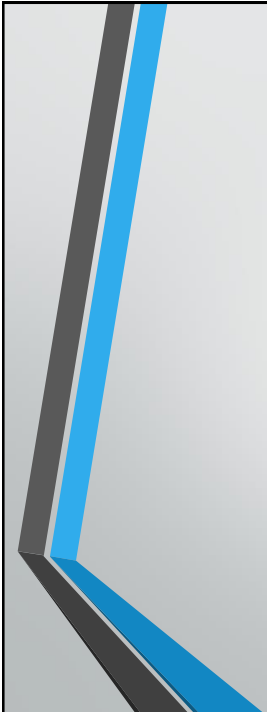




Cours

Algorithmique, Structure de Données et complexité



Plan du cours

Chapitre 1- Les Listes Chainées

- Listes Chainées Simples
- Listes Chainées Doubles
- Listes Chainées Circulaires

Chapitre 2- Les Piles et les Files

Chapitre 3- Les Arbres

Chapitre 1- Les Listes Chainées

Introduction

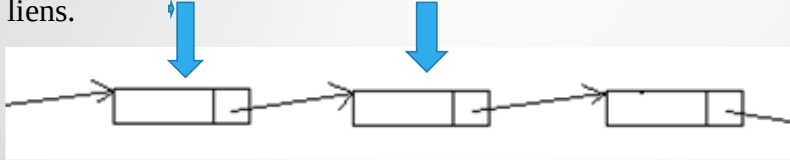
Les structures de données statiques présentent quelques inconvénients:

- taille de la structure fixe
- taille doit être connue à l'avance
- on doit trouver un espace contigu dans la mémoire centrale
=> perte d'espace mémoire.

Une solution est d'utiliser l'allocation dynamique et les structures de données **dynamiques: les listes chaînées**.

Définition

- Une liste chaînée est une structure dynamique formée d'éléments reliés par des liens.



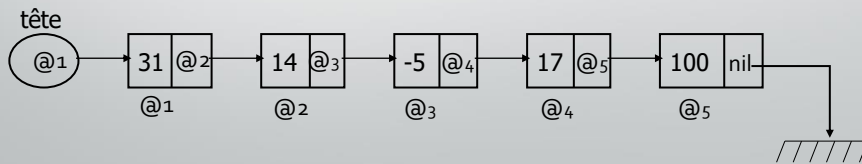
- Les éléments sont éparpillés dans la mémoire centrale.
- La taille de la liste est modifiable: on peut ajouter ou supprimer un élément de la liste.

5

Partie 1 : Les Listes Chainées Simples

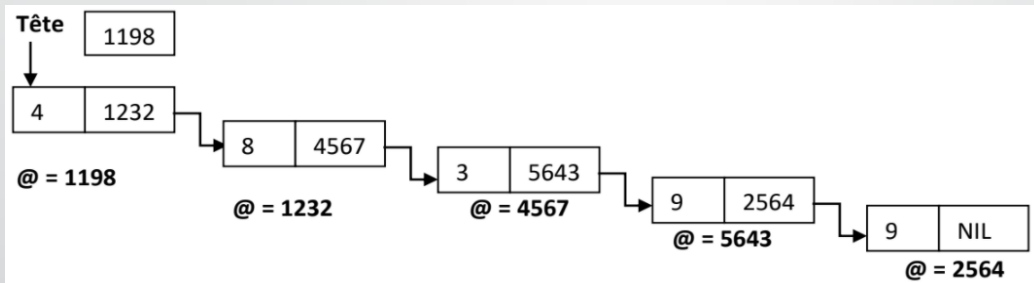
Les listes chaînées simples

- Un élément de la LCS ou nœud est toujours un **enregistrement** (une structure).
 - Les nœuds sont reliés entre eux par des **pointeurs**.
- ➔ un nœud sera composé de deux ou plusieurs champs : un ou plusieurs champ *Valeur* contenant l'information et un champ *Adresse* donnant l'adresse du prochain nœud
- Une liste linéaire chaînée est caractérisée par l'adresse de son premier élément souvent appelé **tête**.
 - Le dernier élément d'une liste chaînée simple contient un pointeur vers *Nil*.



7

Exemple:



- Tête : c'est un pointeur qui contient l'adresse du premier élément de la liste
- Chaque élément de la liste contient deux informations :
 - Une valeur numérique
 - L'adresse de l'élément suivant de la liste

8

Traitements sur les listes chaînées simples:

- Création d'une liste chaînée
- Ajout d'un élément (Allouer)
- Affichage d'une liste
- Modification d'un élément
- Suppression d'un élément (Libérer)
- Recherche d'un élément
- Tri de la liste
- Fusion de 2 listes....

9

Création d'une liste chaînée simple

TYPE

Element = enregistrement
 val : type de valeur
 suivant : pointeur sur Element
 fin enregistrement

Liste = Pointeur sur Element

Procédure CréationListe (var Tête : Liste)

Début

Tête ← Allouer (taille(Element))

Ecrire ("Donnez la valeur de l'élément:")

Lire (X)

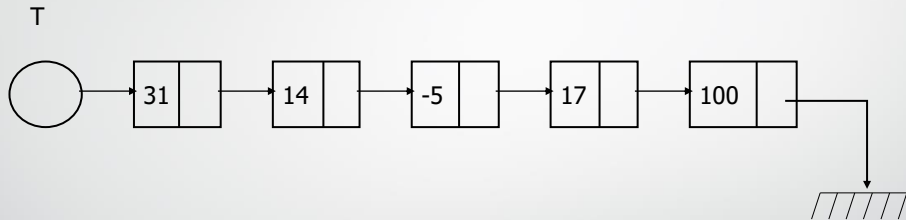
*Tête.val ← X

*Tête.suivant ← nil

Fin

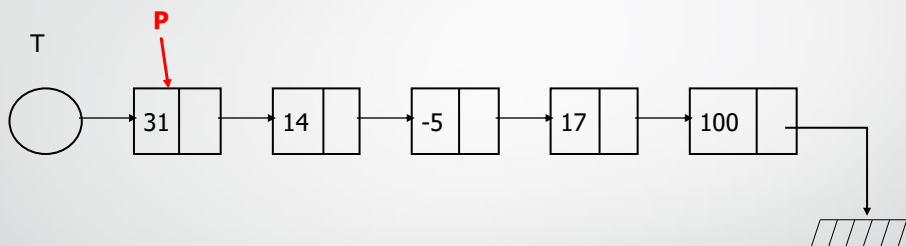
10

Comment parcourir une liste?



11

Comment parcourir une liste?



$P \leftarrow T$

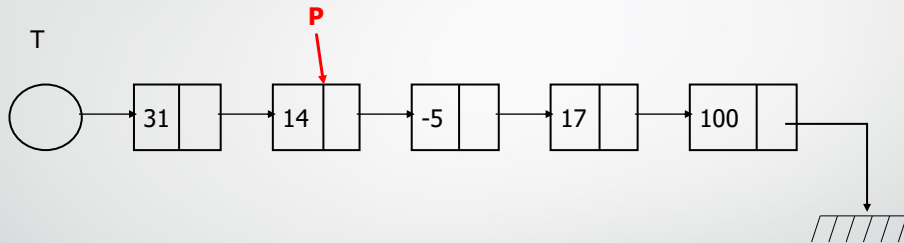
Tant que ($*P.suivant \neq \text{nil}$) faire

$P \leftarrow *P.suivant$

Fin Tant que

12

Comment parcourir une liste?



$P \leftarrow T$

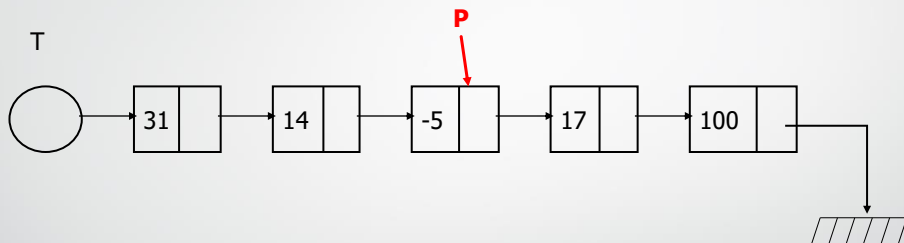
Tant que ($*P.suivant \neq \text{nil}$) faire

$P \leftarrow *P.suivant$

Fin Tant que

13

Comment parcourir une liste?



$P \leftarrow T$

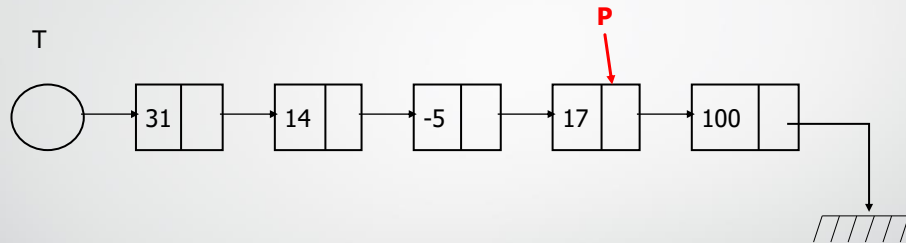
Tant que ($*P.suivant \neq \text{nil}$) faire

$P \leftarrow *P.suivant$

Fin Tant que

14

Comment parcourir une liste?



$P \leftarrow T$

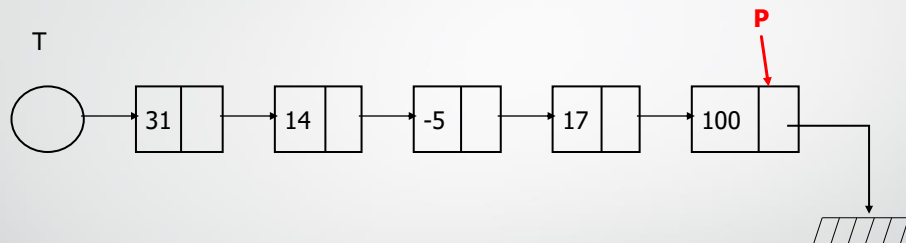
Tant que ($*P.suivant \neq \text{nil}$) faire

$P \leftarrow *P.suivant$

Fin Tant que

15

Comment parcourir une liste?



$P \leftarrow T$

Tant que ($*P.suivant \neq \text{nil}$) faire

$P \leftarrow *P.suivant$

Fin Tant que

16

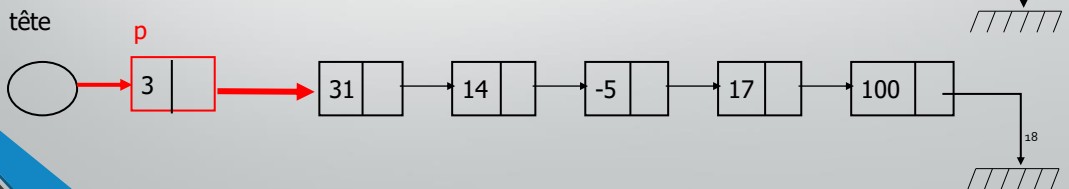
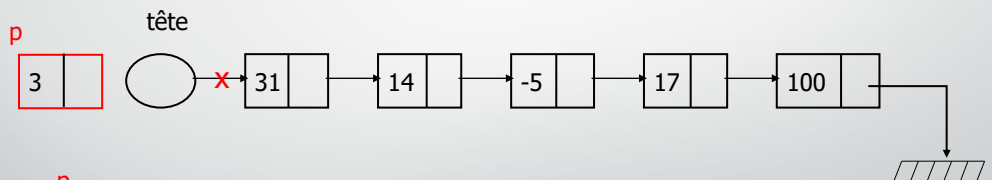
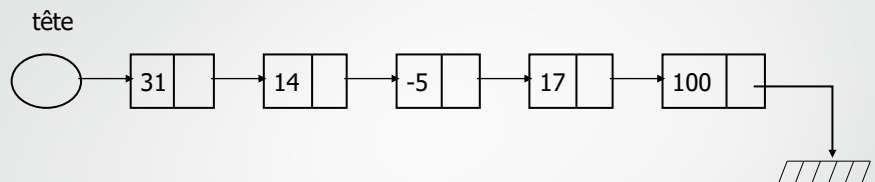
Ajouter un élément dans une liste

Il y a trois possibilités:

- Ajout d'un élément en tête de liste (à gauche)
- Ajout d'un élément au milieu de liste
- Ajout d'un élément en fin de liste (à droite, en queue)

17

Ajout d'un élément en tête de liste



Procédure AJOUT_TETE (var T : Liste)

var

P : Liste

x: entier

DEBUT

Ecrire ("Donner un entier : ")

Lire (x)

P ← Allouer (taille (Elément))

*P.val ← x

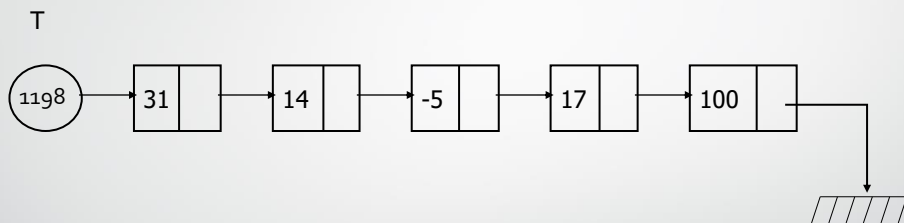
*P.suivant ← T

T ← P

FIN

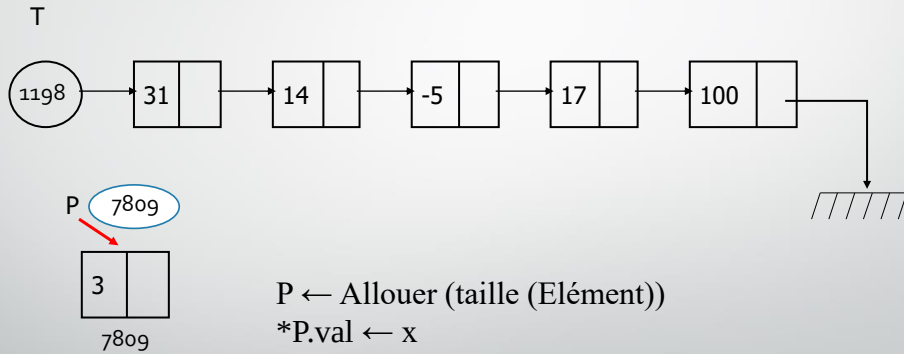
19

Ajout d'un élément en tête de liste



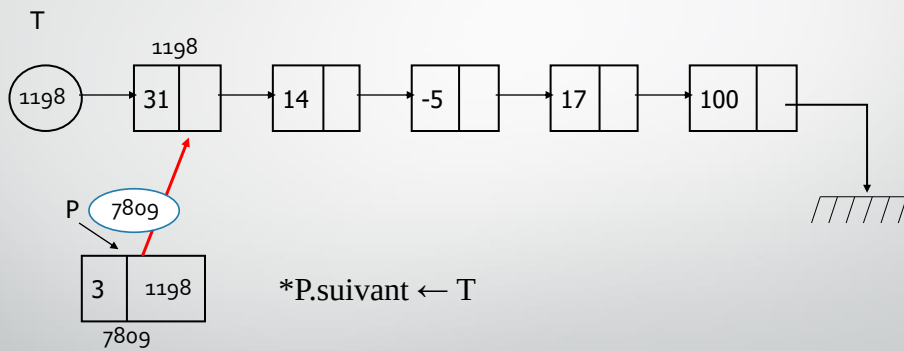
20

Ajout d'un élément en tête de liste



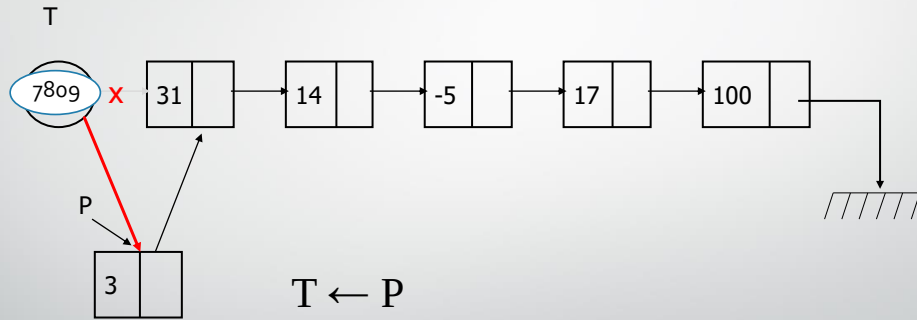
21

Ajout d'un élément en tête de liste



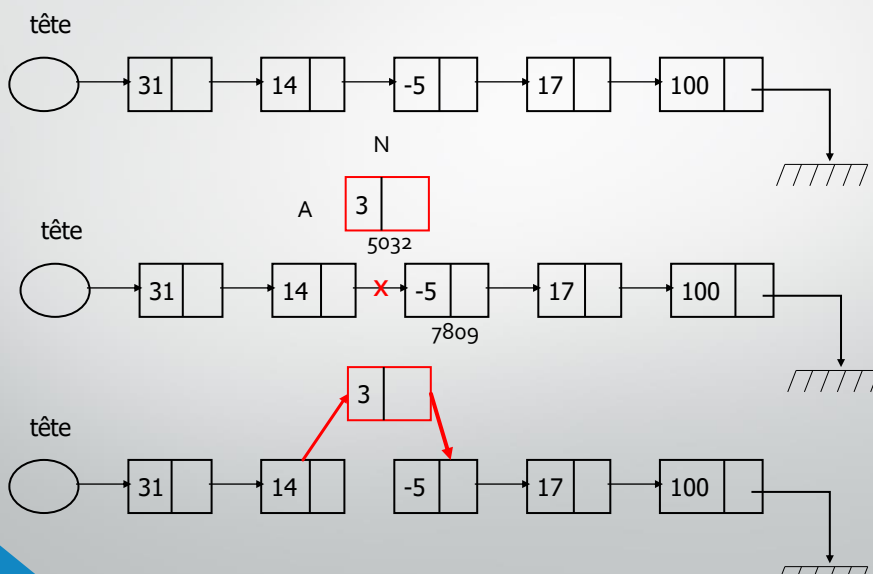
22

Ajout d'un élément en tête de liste



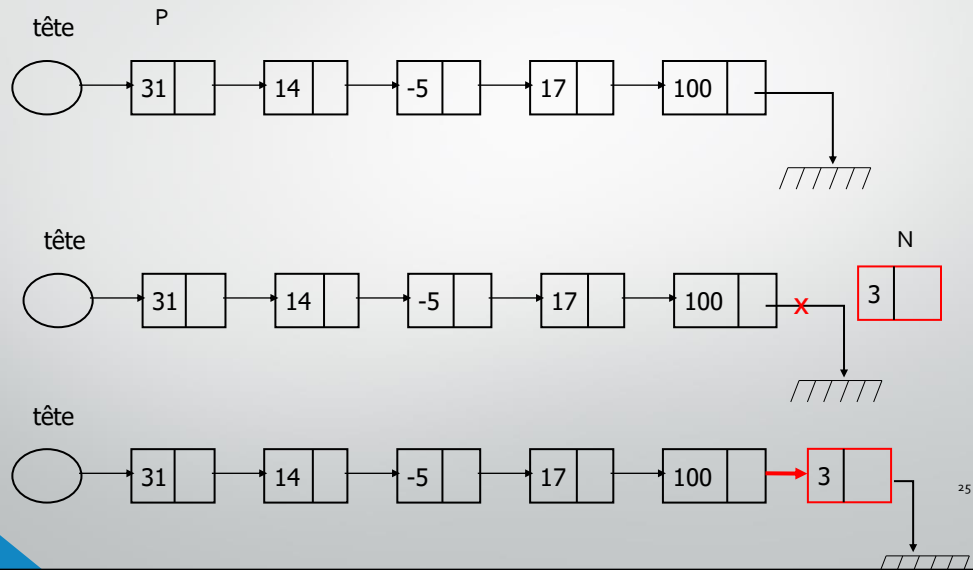
23

Ajout d'un élément au milieu d'une liste



24

Ajout d'un élément en fin de liste (en queue)



Procédure AJOUT_QUEUE (var T:Liste)

var

P, P1 : LISTE

x : entier

DEBUT

Si (T = Nil) **alors**

 AJOUT_TETE (T)

Sinon

 P ← T

 Tant que (*P.suivant ≠ nil) faire

 P ← *P.suivant

 Fin Tant que

Ecrire ("Donner un entier : ")

Lire (x)

P1 ← Allouer (taille (Elément))

*P1.val ← x

*P1.suivant ← NULL

*P.suivant ← P1

Fin Si

FIN

Affichage d'une liste

Procédure AFFICHE_ITER (T : Liste)

var

P : Liste

DEBUT

P ← T

Si (P = NULL) alors

Ecrire ("Liste vide ")

Sinon

Répéter

Ecrire (*P.val)

P ← *P.suivant

Jusqu'à (P = NULL)

Fin Si

FIN

27

Affichage d'une liste

Procédure AFFICHE_REC (T : LISTE)

DEBUT

Si (T ≠ NULL) alors

AFFICHE_REC (*T.suivant)

Ecrire (*T.val)

Fin Si

FIN

28

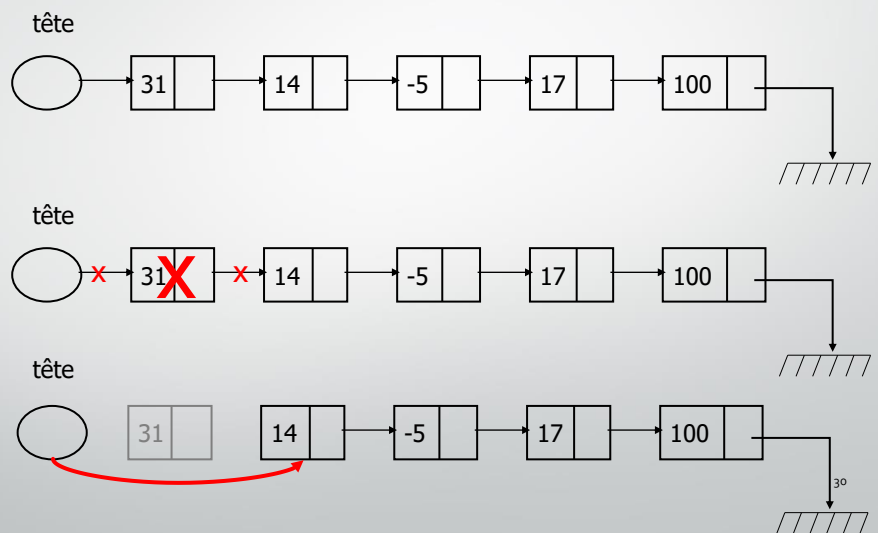
Supprimer un élément dans une liste

Il y a trois possibilités:

- Suppression d'un élément en tête de liste
- Suppression d'un élément au milieu de liste
- Suppression d'un élément en fin de liste

29

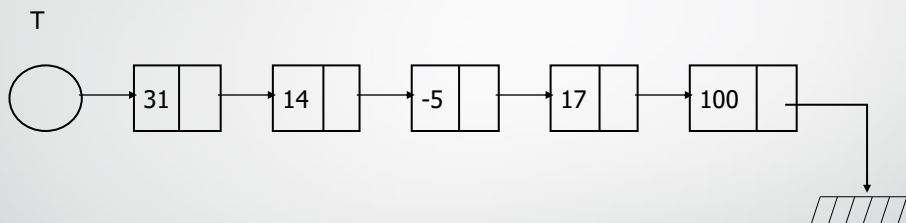
Suppression d'un élément en tête de liste



```
Procédure SUPP_TETE (var T : Liste)
VAR
    P : LISTE
DEBUT
    Si (T ≠ NULL) alors
        P ← T
        T ← *T.suivant
        Libérer (P)
    Fin si
FIN
```

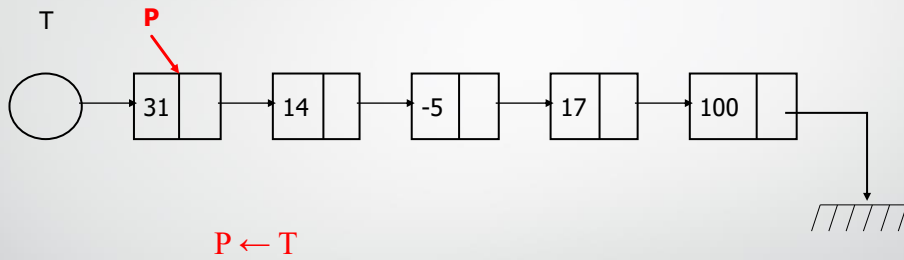
31

Suppression d'un élément en tête de liste



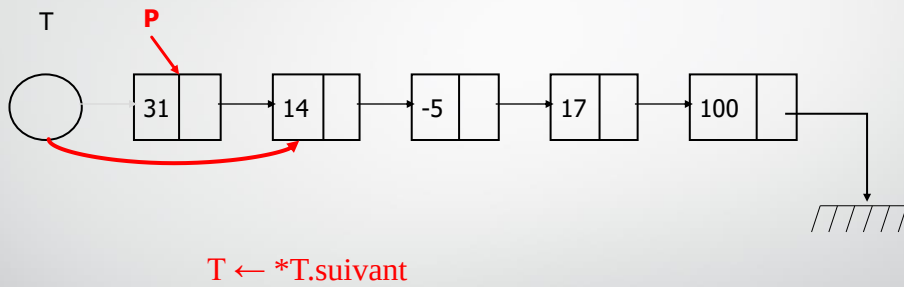
32

Suppression d'un élément en tête de liste



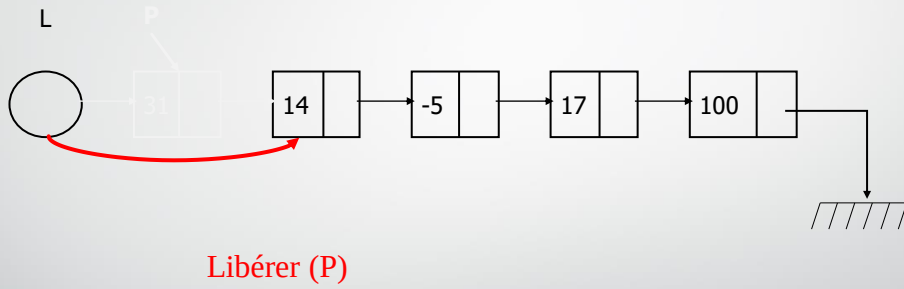
33

Suppression d'un élément en tête de liste



34

Suppression d'un élément en tête de liste



35

Suppression d'un élément en milieu de liste

36

Procédure SupprimElem (var T : Liste; x : entier)

Var

P, avantP : Liste

trouve : booléen

Début

Si (T ≠ nil) **alors**

Si (*T.valeur = x) **alors**

 SUPP_TETE (T)

sinon

 trouve ← faux

 P ← T

TantQue (P ≠ nil) et (trouve = faux) Faire

Si (*P.valeur = x) **Alors**

 trouve ← vrai

sinon

 avantP ← P

 P ← *P.suivant

finSi

FinTantQue

Si (trouve) **alors**

 * avantP.suivant ← *P.suivant

 Libérer (P)

Finsi

FinSi

Sinon

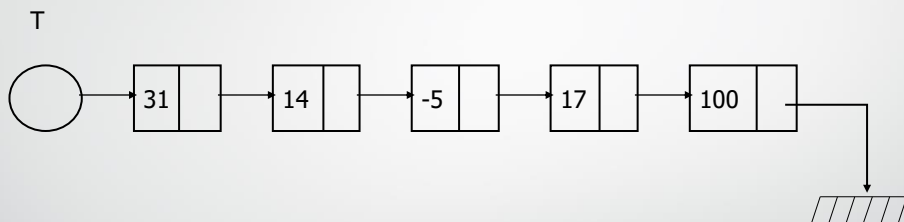
 Ecrire ("Impossible de supprimer liste vide")

FinSi

Fin SupprimElem

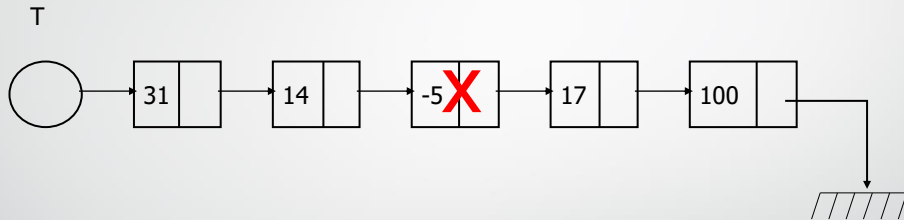
37

Suppression d'un élément en milieu de liste



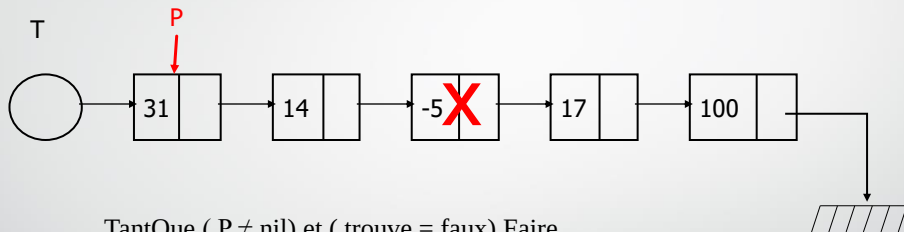
38

Suppression d'un élément en milieu de liste



39

Suppression d'un élément en milieu de liste



TantQue ($P \neq \text{nil}$) et (trouve = faux) Faire

Si ($*P.\text{valeur} = x$) Alors

trouve $\leftarrow$ vrai

sinon

avantP $\leftarrow$ P

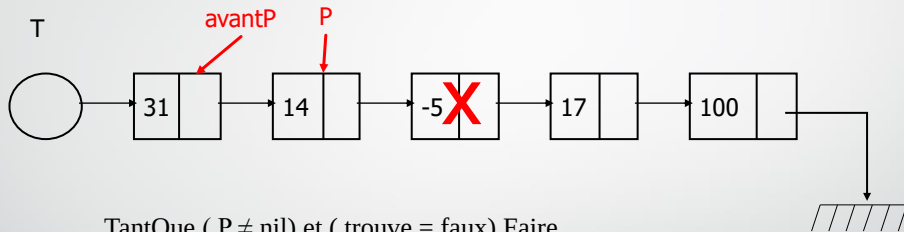
P $\leftarrow$ $*P.\text{suivant}$

finSi

FinTantQue

40

Suppression d'un élément en milieu de liste

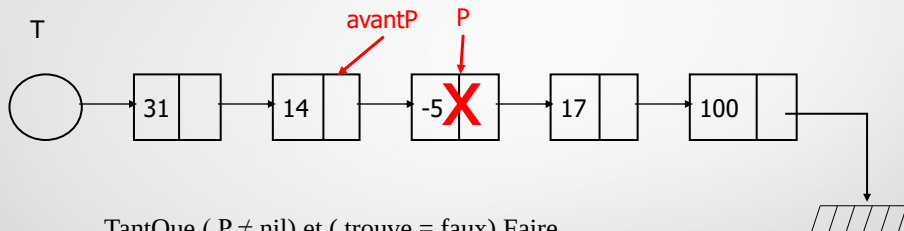


```

TantQue ( P ≠ nil) et ( trouve = faux) Faire
    Si (*P.valeur = x ) Alors
        trouve ← vrai
    sinon
        avantP ← P
        P ← *P.suivant
    finSi
FinTantQue
  
```

41

Suppression d'un élément en milieu de liste

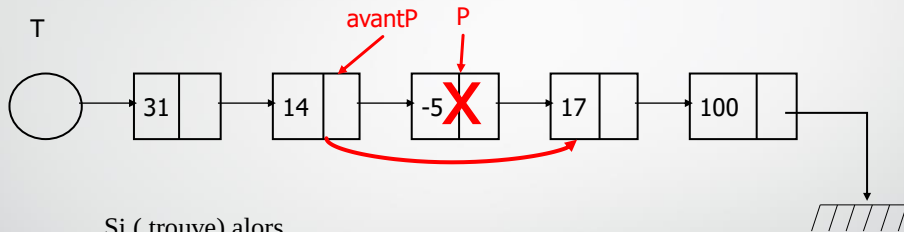


```

TantQue ( P ≠ nil) et ( trouve = faux) Faire
    Si (*P.valeur = x ) Alors
        trouve ← vrai
    sinon
        avantP ← P
        P ← *P.suivant
    finSi
FinTantQue
  
```

42

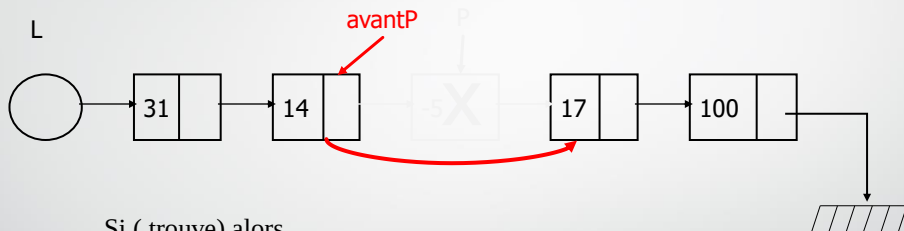
Suppression d'un élément en milieu de liste



Si (trouve) alors
 * avantP.suivant $\leftarrow$ *P.suivant
 Libérer (P)
Finsi

43

Suppression d'un élément en milieu de liste



Si (trouve) alors
 *avantP.suivant $\leftarrow$ *P.suivant
 Libérer (P)
Finsi

44

Partie 2 : Les Listes Chainées Doubles

Les listes chaînées doubles

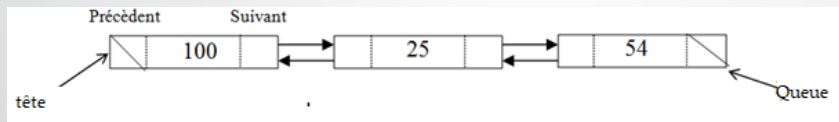
Dans la liste chaînée simple, le parcours ne peut se faire que dans un seul sens (de la tête vers la queue) ce qui peut engendrer parfois une augmentation dans les temps d'exécution ou encore des complications au niveau des instructions.

Pour contourner cet inconvénient, et permettre un parcours aussi bien dans un sens que dans l'autre, nous allons construire des **listes doublement chaînées** (listes **bilatères** ou encore **bidirectionnelles**).

46

Définition

- Une liste doublement chaînée ou bidirectionnelle est telle que chaque élément de la liste contient un champ qui pointe sur l'élément précédent et un champ qui pointe sur l'élément suivant. De ce fait, la liste peut être parcourue dans les deux sens.
- En plus de pointeur tête, elle admet un **pointeur Queue** qui permet de la parcourir à partir de la fin.



- Tous les éléments de la liste (sauf le premier) contiennent l'adresse de l'élément précédent
- Tous les éléments de la liste (sauf le dernier) contiennent l'adresse de l'élément suivant
- Le champ précédent du premier élément de la liste et le champ suivant du dernier élément de la liste contiennent la valeur Nil
- La liste est considérée **vide** lorsque **tête = queue = nil** (tete =nil et queue = nil)
- La liste contient **un seul élément** lorsque **tête = queue** et **tete <> nil**

47

Les listes chaînées doubles

- Un élément de la LCD est un **enregistrement**. Les éléments sont reliés entre eux par des **adresses**.
- ➔ un élément sera composé de trois champs ou plus :
 - un ou plusieurs champ *Valeur* contenant l'information,
 - un champ *Adresse suivant* contenant l'adresse du prochain élément
 - et un champ adresse précédent contenant l'adresse de l'élément précédent
- Une liste bidirectionnelle est caractérisée par l'adresse de son premier élément souvent appelé **tête** et l'adresse du dernier élément appelé **queue**.

48

Traitements sur les listes chaînées doubles:

- Création d'une liste chaînée double
- Ajout d'un élément (Allouer)
- Suppression d'un élément (Libérer)
- Modification d'un élément
- Affichage d'une liste
- Recherche d'un élément
-

49

Création d'une liste chaînée double

```

TYPE
ElemD = enregistrement
    prec : pointeur su ElemD
    val : type de valeur // entier , réel , caractère...
    suiv : pointeur sur ElemD
fin enregistrement

```

```
ListeBi = Pointeur sur ElemD
```

```
Procédure CréationListe (var Tête, var Queue : ListeBi) // initialement tete=nil et queue =nil
```

```
Début
```

```
    Tête ← Allouer (taille(ElemD))
```

```
    Ecrire ("Donnez la valeur de l'élément:")
```

```
    Lire ( X ) //par exemple x= 46
```

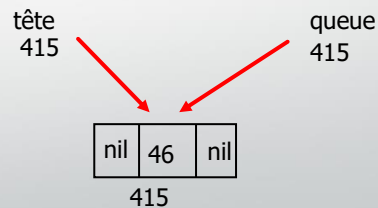
```
    *Tête.val ← X
```

```
    *Tête.suiv ← nil
```

```
    *Tête.prec ← nil
```

```
    Queue ← Tête
```

```
Fin
```



50

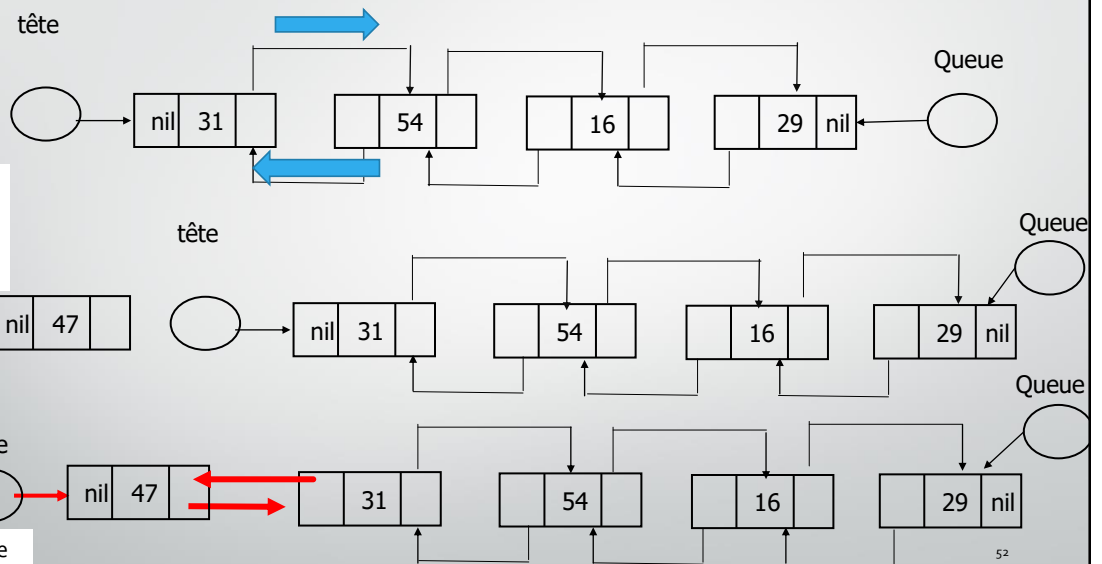
Ajouter un élément dans une liste

Il y a trois possibilités:

- Ajout d'un élément en tête de liste (à gauche)
- Ajout d'un élément au milieu de liste
- Ajout d'un élément en fin de liste (à droite, en queue)

51

Ajout d'un élément en tête de liste



52

Procédure AJOUT_TETE (var T , var Q : ListeBi)

var

P : ListeBi

x: entier

DEBUT

Ecrire ("Donner un entier : ")

Lire (x)

P ← Allouer (taille (ElemD))

*P.val ← x

*P.prec ← nil

*P.suiv ← T

si (T <> nil) alors

 *T.prec ← P

finsi

T ← P

si (Q=nil) alors

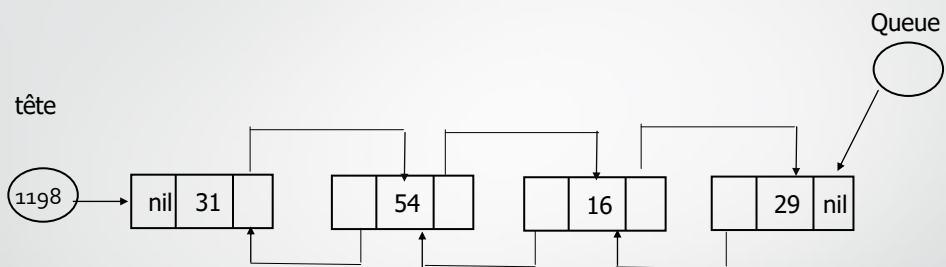
 Q ← T

FinSi

FIN

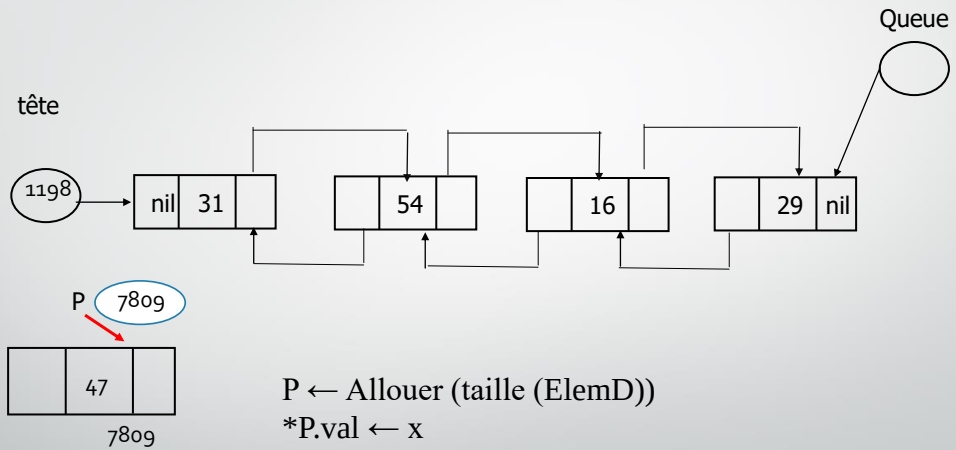
53

Ajout d'un élément en tête de liste



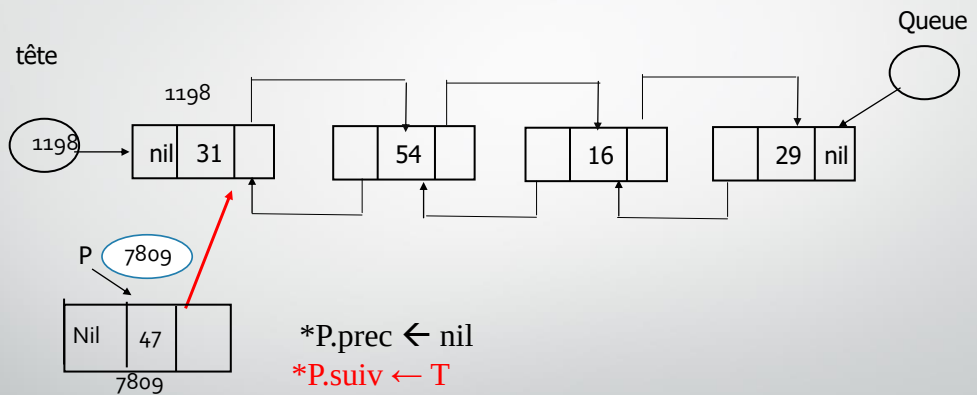
54

Ajout d'un élément en tête de liste



55

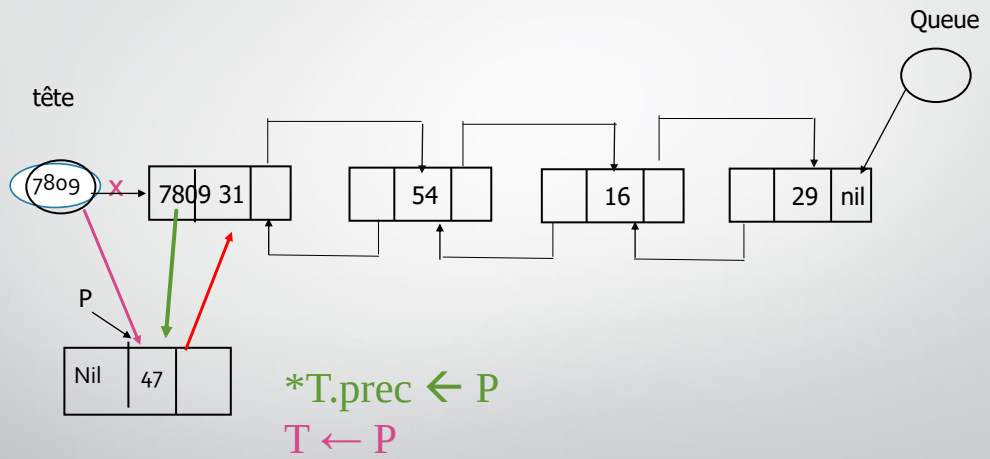
Ajout d'un élément en tête de liste



56

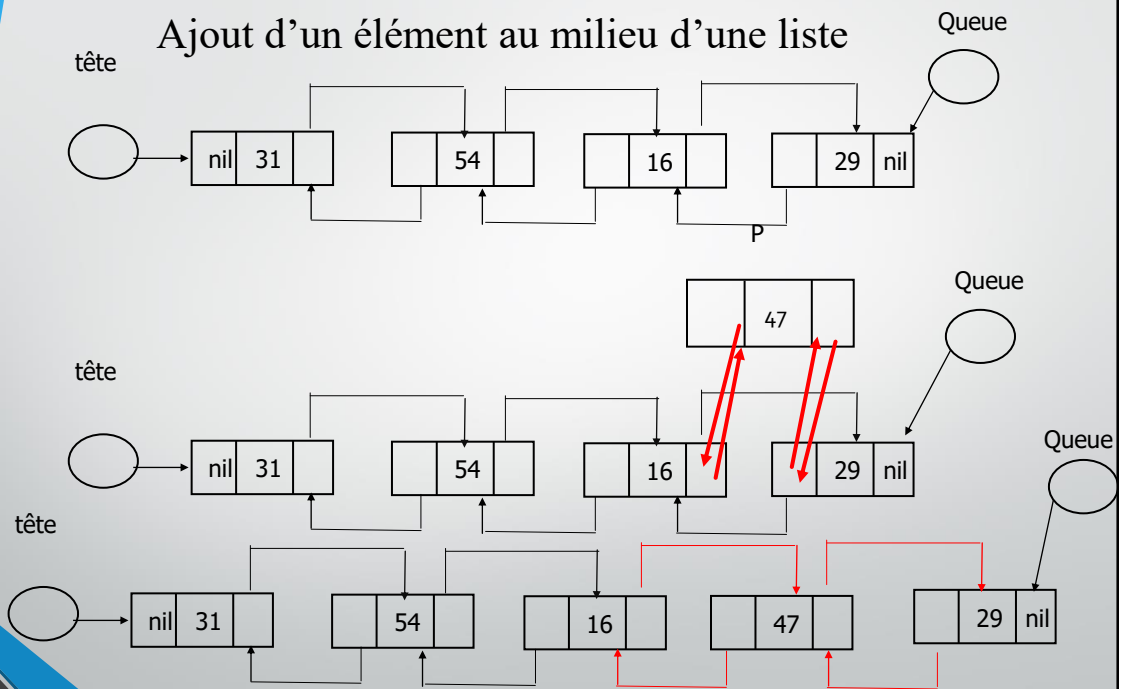
47

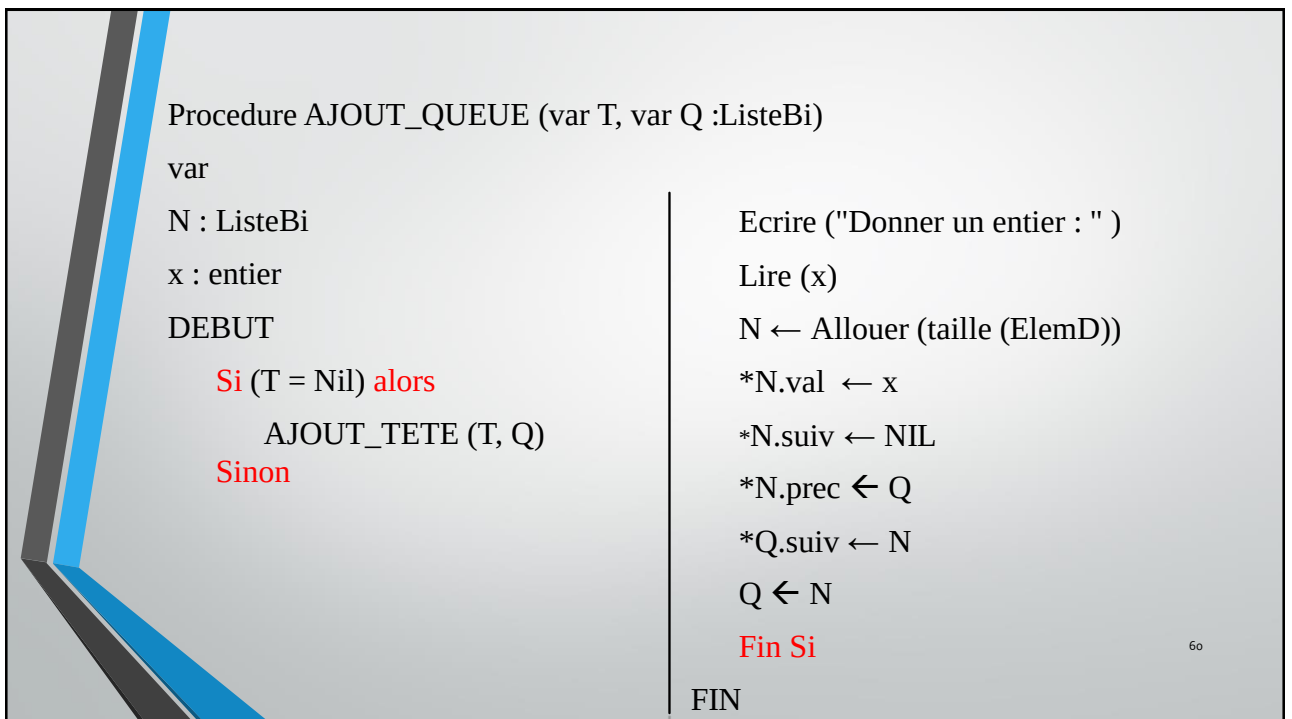
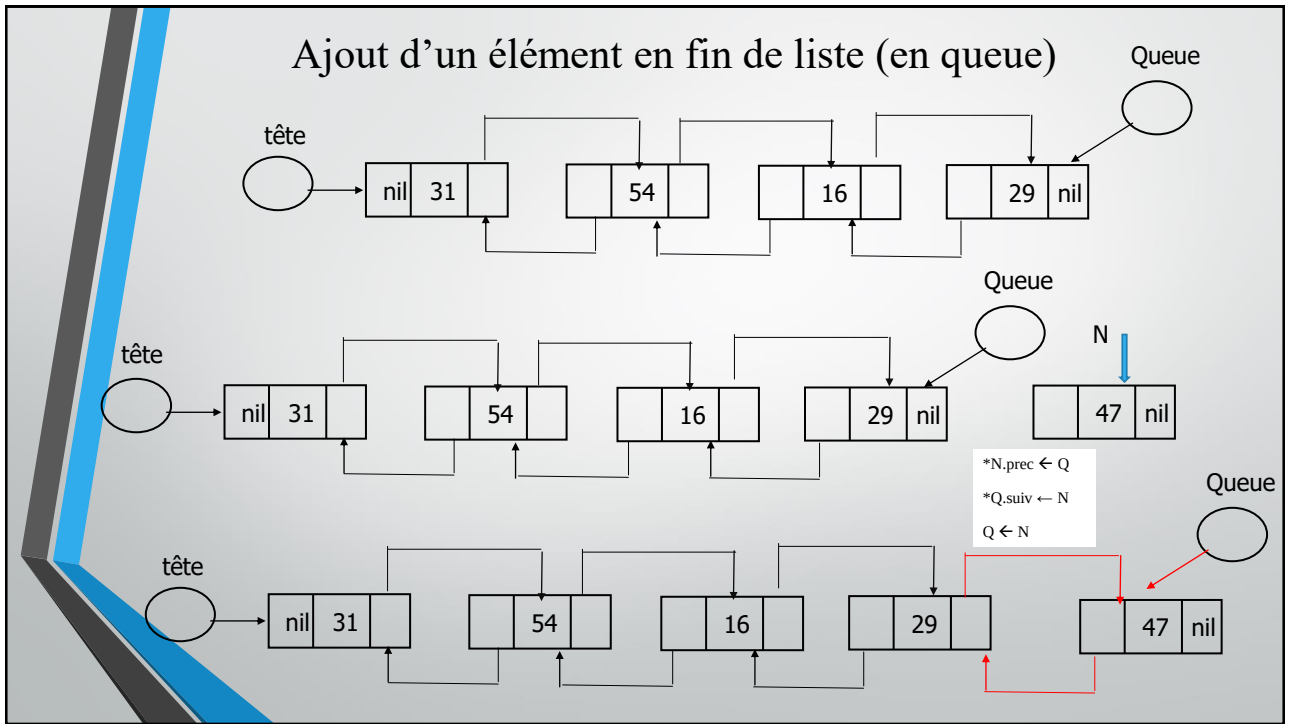
Ajout d'un élément en tête de liste



57

Ajout d'un élément au milieu d'une liste

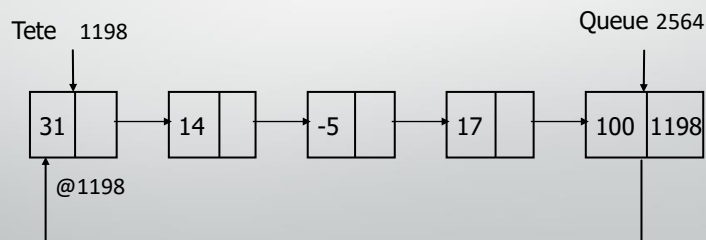




Partie 3 : Les Listes Chainées Circulaires

Les listes chaînées circulaires

- Structure très proche de celle de la liste chaînée simple, mais avec deux différences
- Le dernier élément de la liste pointe sur le premier
- Il existe parfois un pointeur additionnel *queue* qui pointe vers le dernier élément de la liste



Les listes chaînées circulaires

Même déclaration que LCS

```
TYPE
  Element = enregistrement
    val : type de valeur
    suivant : pointeur sur Element
  fin enregistrement
```

```
Liste = Pointeur sur Element
```

63

Ajout d'un élément en tête de liste

Procédure AJOUT_TETE (var Tete , var Queue : Liste, x : entier)

var

P : Liste

DEBUT

P ← Allouer (taille (Element))

*P.val ← x

si (Tete = nil) alors

Tete ← P

Queue ← P

*p.suivant ← Tete

sinon

*p.suivant ← Tete

Tete ← p

*Queue.suivant ← p

finsi

FIN

64

Ajout d'un élément en fin de liste (en queue)

Procédure AJOUT_Queue (var Tete , var Queue : Liste, x : entier)

var

P : Liste

DEBUT

P ← Allouer (taille (Element))

*P.val ← x

si (Tete = nil) alors

Tete ← P

Queue ← P

*p.suivant ← Tete

sinon

*p.suivant ← Tete // ou bien *p.suivant ← *Queue.suivant

*Queue.suivant ← p

Queue ← p

finsi

FIN

65

Déterminer la longueur d'une liste circulaire

Fonction DetLongCirculaire (Tete , Queue : Liste) : entier

var

P : Liste

L : entier

DEBUT

L ← 0

si (Tete <> nil) alors

P ← Tete

Répéter

L ← L + 1

p ← *p.suivant

Jusqu'à (p = Tete)

Finsi

Retourner (L)

FIN

66